

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)

Assessment Tool Weightage: 20%

QUESTIONS:

**Duration :60 Mins
On class
Total Marks:50**

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I _____ SOFIA R.V. - 192511055 _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: SOFIA RV

Annexure A

ERROR ANALYSIS TASK

Q1. The following C program is intended to perform string processing, function calls, array manipulation, conditional execution, pointer operations, switch-case selection, and loop execution. However, the program contains several errors introduced during different phases of compilation, including lexical, syntax, semantic, logical, and runtime errors.

Given Program

```
#include <stdio.h>
#include <string.h>

int findLength(char str[])
{
    return strlen(str);
}

int main()
{
    char name[20] = "Compiler"

    int count;

    count = findLength();

    if(count > 5
    {
        printf("Length is greater than 5\n");
    }

    int marks[4] = {85, 90, 75, 80};

    printf("%d\n", marks[4]);

    int choice = 2;

    switch(choice)
    {
        case 1:
            printf("Option 1\n");
            break;
```

```

        case 2
            printf("Option 2\n");
            break;

        default:
            printf("Invalid Choice\n");
    }

    int *ptr;
    printf("%d\n", *ptr);

    int total = 100;
    int students = 0;

    float average = total / students;

    int i = 1;

    while(i <= 5)
    {
        printf("%d\n", i);
    }

    return 0;
}

```

Tasks

- (a) Identify all errors in the program. Mention the line number, error type, and reason for each error. (5 Marks)

Line No.	Error	Error Type	Reason
12	Missing ; after "Compiler"	Syntax Error	Every statement must end with a semicolon.
16	findLength() called without argument	Semantic Error	The function requires one argument (name).
18	Missing) in if(count > 5	Syntax Error	Closing parenthesis is missing.
29	Missing : after case 2	Syntax Error	Every case label must end with a colon.

24	<code>marks[4] accessed</code>	Logical/Runtime Error	Array size is 4, so valid indices are 0–3.
40	Dereferencing uninitialized pointer <code>*ptr</code>	Runtime Error	Pointer does not point to a valid memory location.
45	Division by zero (<code>students = 0</code>)	Runtime Error	Division by zero causes program failure.

(b) Classify the identified errors as Lexical, Syntax, Semantic, Logical, or Runtime errors in a suitable table. (5 Marks)

Error	Category
Missing semicolon	Syntax Error
Missing function argument	Semantic Error
Missing <code>)</code> in <code>if</code>	Syntax Error
Missing <code>:</code> after <code>case 2</code>	Syntax Error
Array index out of bounds	Logical / Runtime Error
Uninitialized pointer	Runtime Error
Division by zero	Runtime Error
Infinite loop	Logical Error

(c) Explain at which phase of compilation each category of error is detected. Also mention which errors are detected only during execution. (4 Marks)

Error Category	Detected During
Lexical Error	Lexical Analysis Phase
Syntax Error	Syntax Analysis (Parser)
Semantic Error	Semantic Analysis
Logical Error	During Testing/Execution
Runtime Error	During Program Execution

Errors detected only during execution

- Dereferencing an uninitialized pointer
- Division by zero
- Array index out of bounds
- Infinite loop

(d) Rewrite the program after correcting all the errors so that it executes successfully. (4 Marks)

```
#include <stdio.h>
#include <string.h>

int findLength(char str[])
{
    return strlen(str);
}

int main()
{
    char name[20] = "Compiler";

    int count;

    count = findLength(name);

    if(count > 5)
    {
        printf("Length is greater than 5\n");
    }

    int marks[4] = {85, 90, 75, 80};

    printf("%d\n", marks[3]);

    int choice = 2;

    switch(choice)
    {
        case 1:
            printf("Option 1\n");
            break;

        case 2:
            printf("Option 2\n");
            break;

        default:
            printf("Invalid Choice\n");
    }

    int value = 50;
    int *ptr = &value;

    printf("%d\n", *ptr);

    int total = 100;
```

```

int students = 5;

float average = (float)total / students;

printf("Average = %.2f\n", average);

int i = 1;

while(i <= 5)
{
    printf("%d\n", i);
    i++;
}

return 0;
}

```

```

1 #include <stdio.h>
2 #include <string.h>
3 int findlength(char str[])
4 {
5     return strlen(str);
6 }
7 int main()
8 {
9     char name[] = "Compiler";
10    int count;
11    count = findlength(name);
12    if(count > 5)
13    {
14        printf("length is greater than 5\n");
15    }
16    int marks[] = {85, 90, 75, 80};
17    printf("%d\n", marks[1]);
18    int choice = 2;
19    switch(choice)
20    {
21        case 1:
22            printf("Option 1\n");
23            break;
24        case 2:
25            printf("Option 2\n");
26            break;
27        default:
28            printf("Invalid Choice\n");
29    }
30    int value = 50;
31    int *ptr = &value;
32    printf("%d\n", *ptr);
33 }

```

Option 2
50
Average = 20.00
1
2
3
4
5

(e) Suggest any three compiler features or warnings that can help detect such programming errors before execution. (2 Marks)

Syntax Error Checking

- Detects missing semicolons, brackets, and colons.

Compiler Warning Flags (-Wall, -Wextra)

- Warns about uninitialized variables, unused variables, and possible coding mistakes.

Static Code Analysis

- Detects potential runtime issues such as null/uninitialized pointers, array bounds violations, and division by zero before execution.

Q2. The following C program is intended to perform function calls, array processing, conditional execution, switch-case selection, and loop operations. However, several syntax errors have been intentionally introduced by violating the grammatical rules of the C language. These errors include missing delimiters, unmatched parentheses, incorrect control structures, malformed function definitions, and incomplete statements. Such errors are normally detected during the syntax analysis (parsing) phase of a compiler.

Carefully examine the program and answer the questions that follow.

Given Program

```
#include <stdio.h>

int maximum(int x, int y
{
    if(x > y)
        return x;
    else
        return y;
}

int main()
{
    int marks[5] = {70, 80, 90, 85, 95};

    int max = maximum(marks[0], marks[1]);

    switch(max)
    {
        case 80:
            printf("Maximum is 80\n");
            break;

        case 90
            printf("Maximum is 90\n");
            break;

        default:
            printf("Other Value\n");
    }

    do
    {
        printf("%d\n", max);
        max--;
    }
```

```

while(max > 75);

if(max == 75)
    printf("Limit Reached\n"
}

return 0;

```

Tasks

(a) Identify all syntax errors in the program. Mention the line number and reason for each error. (4 Marks)

Line No.	Syntax Error	Reason
3	int maximum(int x, int y	Missing closing parenthesis) before {.
18	printf("Maximum is 80\n")	Missing semicolon ;.
21	case 90	Missing colon : after case 90.
31	while(max > 75);	Missing closing brace } before while in do-while loop.
34	printf("Limit Reached\n"	Missing closing parenthesis) and semicolon ;.
35	}	Unmatched brace due to previous syntax error.
38	End of program	Missing closing brace } for main() function.

(b) Classify the identified syntax errors under the appropriate categories and present them in a table. (3 Marks)

Error Category	Error Category
Missing) in function definition Parenthesis Error	Missing) in function definition Parenthesis Error
Missing ; after printf() Missing Delimiter	Missing ; after printf() Missing Delimiter
Missing : after case Switch Statement Error 90	Missing : after case Switch Statement Error 90
Incorrect do-while syntax Loop Structure Error	Incorrect do-while syntax Loop Structure Error
Missing) and ; in printf() Function Call Syntax Error	Missing) and ; in printf() Function Call Syntax Error
Missing closing brace } Brace Matching Error	Missing closing brace } Brace Matching Error

(c) Correct the syntax errors and rewrite the complete program. (4 Marks)

```
#include <stdio.h>
```

```
int maximum(int x, int y)
```

```
{
    if(x > y)
        return x;
    else
        return y;
}
```

```
int main()
```

```
{
    int marks[5] = {70, 80, 90, 85, 95};

    int max = maximum(marks[0], marks[1]);

    switch(max)
    {
        case 80:
            printf("Maximum is 80\n");
            break;

        case 90:
```

```

        printf("Maximum is 90\n");
        break;

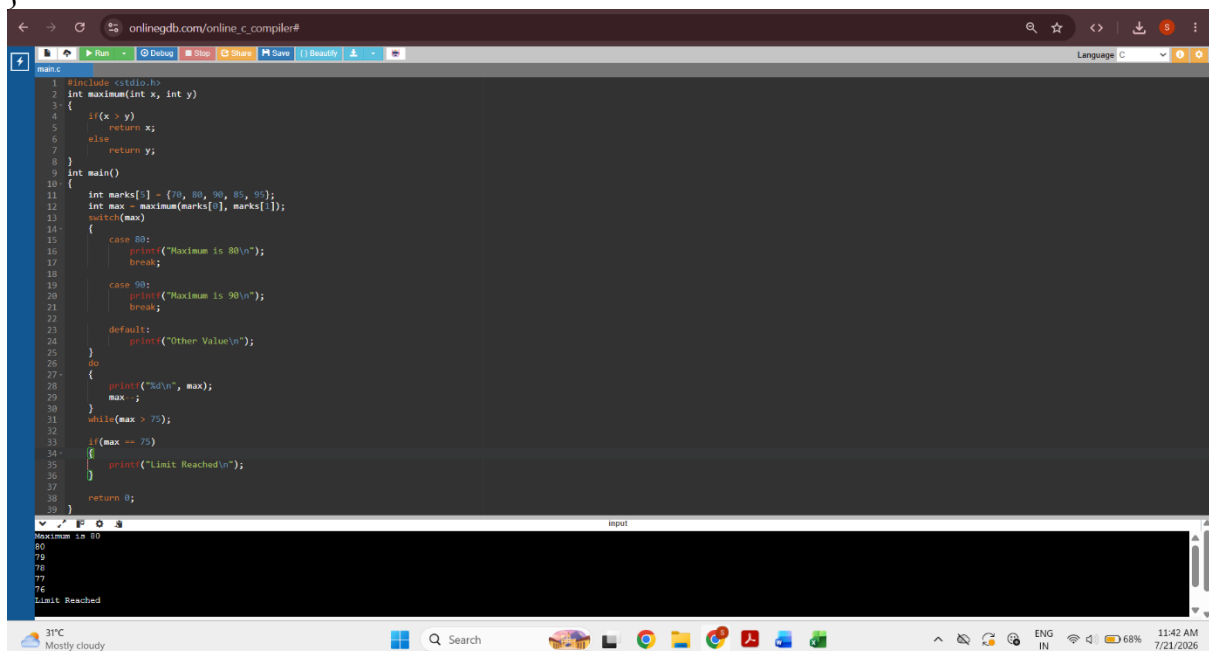
    default:
        printf("Other Value\n");
    }

do
{
    printf("%d\n", max);
    max--;
}
while(max > 75);

if(max == 75)
{
    printf("Limit Reached\n");
}

return 0;
}

```



The screenshot shows a web browser window with the URL `onlinegdb.com/online_c_compiler#`. The code editor contains the following C program:

```

1 #include <stdio.h>
2 int maximum(int x, int y)
3 {
4     if(x > y)
5         return x;
6     else
7         return y;
8 }
9 int main()
10 {
11     int marks[] = {70, 80, 90, 85, 95};
12     int max = maximum(marks[0], marks[1]);
13     switch(max)
14     {
15         case 80:
16             printf("Maximum is 80\n");
17             break;
18
19         case 90:
20             printf("Maximum is 90\n");
21             break;
22
23         default:
24             printf("Other Value\n");
25     }
26     do
27     {
28         printf("%d\n", max);
29         max--;
30     }
31     while(max > 75);
32
33     if(max == 75)
34     {
35         printf("Limit Reached\n");
36     }
37
38     return 0;
39 }

```

The output window shows the following results:

```

Maximum is 80
80
79
78
77
76
Limit Reached

```

The bottom of the screenshot shows a Windows taskbar with the date and time 11:42 AM 7/21/2026 and a battery level of 68%.

(d) Explain how the parser detects syntax errors and how error recovery enables further parsing. (2 Marks)

The **parser** checks whether the program follows the grammar rules of the C language. It detects syntax errors such as missing semicolons, unmatched parentheses, missing braces, and invalid control statements.

Error Recovery allows the parser to skip the incorrect part of the program and continue parsing the remaining code so that multiple syntax errors can be reported in a single compilation.

(e) Suggest any two improvements that can help a compiler generate better syntax error messages. (2 Marks)

1. Precise Line and Column Numbers

- Helps programmers quickly locate the error.

2. Meaningful Error Suggestions

- Example: "Expected ';' before 'break'" instead of simply "Syntax Error".

Q3. The following C program is intended to perform factorial computation, function calls, array processing, pointer manipulation, conditional execution, switch-case selection, and loop operations. However, the program contains several errors introduced during different phases of compilation and execution, including lexical, syntax, semantic, intermediate code generation, code optimization, and runtime errors.

Carefully examine the program and answer the questions that follow.

Given Program

```
#include <stdio.h>
```

```
int factorial(int n)
```

```
{
```

```
    if(n <= 1)
```

```
        return 1;
```

```
    return n * factorial(n - 1);
```

```
}
```

```
int main()
```

```
{
```

```
    int @num = 5;
```

```
int value = 10

int result;

result = factorial();

if(result >= 100)
{
    printf("Large Number\n");
}

switch(result)
{
    case 24:
        printf("Factorial of 4\n");
        break;

    case 120
        printf("Factorial of 5\n");
        break;

    default:
        printf("Other Value\n");
}

int numbers[5] = {2,4,6,8,10};
printf("%d\n", numbers[5]);

int *ptr;
printf("%d\n", *ptr);

int a = 50;
int b = 0;
int c = a / b;

int sum = result + 0 + value;

int i = 1;
while(i <= 5)
{
    printf("%d\n", i);
}

return 0;
}
```

Tasks

(a) Identify all the errors in the program. Mention the line number, error type, and reason for each error. (4 Marks)

Line No.	Error	Error Type	Reason
13	<code>int @num = 5;</code>	Lexical Error	@ is an invalid character in an identifier.
15	<code>int value = 10</code>	Syntax Error	Missing semicolon ;.
19	<code>factorial();</code>	Semantic Error	Function requires one argument.
31	<code>case 120</code>	Syntax Error	Missing colon :.
39	<code>numbers[5]</code>	Runtime Error	Array index out of bounds.
42	<code>*ptr</code>	Runtime Error	Pointer is uninitialized.
46	<code>a / b</code>	Runtime Error	Division by zero.
48	<code>result + 0 + value</code>	Code Optimization	Adding 0 is unnecessary and can be optimized.
51-54	<code>while(i <= 5)</code>	Logical / Runtime Error	Infinite loop because i is never incremented.

(b) Classify the identified errors according to the appropriate compiler phase (Lexical, Syntax, Semantic, Intermediate Code Generation, Code Optimization, and Runtime) using a suitable table. (4 Marks)

Error	Compiler Phase
Invalid identifier (@num)	Lexical Analysis
Missing semicolon	Syntax Analysis
Missing function argument	Semantic Analysis
Missing : after <code>case 120</code>	Syntax Analysis
<code>result + 0</code>	Code Optimization
Array index out of bounds	Runtime
Uninitialized pointer	Runtime
Division by zero	Runtime
Infinite loop	Runtime / Logical

(c) Explain how the compiler processes the program by identifying the errors detected during each compilation phase and those detected only at runtime. (3 Marks)

- **Lexical Analysis**

Detects invalid tokens such as @num.

- **Syntax Analysis**

Detects missing semicolons, missing colons, and incorrect program structure.

- **Semantic Analysis**

Checks function calls, variable declarations, and data types. It detects the missing argument in factorial().

- **Intermediate Code Generation**

Generates intermediate representation after successful lexical, syntax, and semantic analysis.

- **Code Optimization**

Optimizes unnecessary expressions such as result + 0.

- **Runtime**

Detects errors that occur only during execution, such as division by zero, invalid array access, uninitialized pointer dereference, and infinite loops.

(d) Rewrite the complete corrected program so that it compiles and executes without errors. (2 Marks)

```
#include <stdio.h>
```

```
int factorial(int n)
{
    if(n <= 1)
        return 1;

    return n * factorial(n - 1);
}
```

```
int main()
{
    int num = 5;

    int value = 10;

    int result;

    result = factorial(num);
```

```
if(result >= 100)
{
    printf("Large Number\n");
}

switch(result)
{
    case 24:
        printf("Factorial of 4\n");
        break;

    case 120:
        printf("Factorial of 5\n");
        break;

    default:
        printf("Other Value\n");
}

int numbers[5] = {2,4,6,8,10};
printf("%d\n", numbers[4]);

int data = 50;
int *ptr = &data;
printf("%d\n", *ptr);

int a = 50;
int b = 5;
int c = a / b;

int sum = result + value;

int i = 1;
while(i <= 5)
{
    printf("%d\n", i);
    i++;
}

return 0;
}
```

```
1 #include <stdio.h>
2 int factorial(int n)
3 {
4     if(n <= 1)
5         return 1;
6
7     return n * factorial(n - 1);
8 }
9 int main()
10 {
11     int num = 5;
12     int value = 10;
13     int result;
14     result = factorial(num);
15     if(result == 100)
16     {
17         printf("Large Number\n");
18     }
19     switch(result)
20     {
21         case 24:
22             printf("Factorial of 4\n");
23             break;
24         case 120:
25             printf("Factorial of 5\n");
26             break;
27         default:
28             printf("Other Value\n");
29     }
30     int numbers[] = {2,4,6,8,10};
31     printf("%d\n", numbers[1]);
32     int data = 50;
33     int ptr = &data;
34     printf("%d\n", ptr);
35     int a = 50;
36     int b = 5;
37     int c = a / b;
38     int sum = result + value;
39     int i = 1;
40     while(i <= 5)
41     {
42         printf("%d\n", 1);
43     }
44 }
```

Large Number
Factorial of 5
10
30
1
2
3
4
5

(e) Suggest any two improvements in compiler design or optimization techniques that could detect or prevent such errors before program execution. (2 Marks)

1. Static Code Analysis

- Detects possible runtime errors such as null pointers, array bounds violations, and division by zero before execution.

2. Advanced Compiler Warnings (-Wall, -Wextra)

- Warns about uninitialized variables, suspicious code, and potential logical errors, helping programmers fix issues early.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand